SPHERE WEB AUTOMATION

Public Page Script Step-by-Step Walkthrough

A section-by-section, element-by-element map of how `tests/PublicPage.spec.ts` verifies the Sphere public home page — every locator, scroll, assertion and bilingual fallback, with page-anatomy and execution diagrams.

Script tests/PublicPage.spec.ts · 823 lines · 64 tests in 13 element groups + page-wide checks

Target https://sphere.aastrika.org/public/home (no login required)

Run npm run run:public

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	Configuration and inputs	URL normalisation, runner settings
3	The bilingual engine	Diagram — how tr() matches Hindi or English
4	The execution model	Diagram — one fresh page per test, four assertion patterns
5	Page anatomy	Diagram — all 13 groups mapped onto the page
6	Group 1 — Header	2 tests
7	Group 2 — Hero Banner	7 tests
8	Group 3 — Stats Row	4 tests
9	Group 4 — Top Courses	6 tests
10	Group 5 — CNE Courses	9 tests
11	Group 6 — Trusted By	7 tests
12	Group 7 — How Sphere Works	5 tests
13	Groups 8 & 9 — Transform CTA, Initiative	2 + 4 tests
14	Groups 10 & 11 — Footer: Quick Links, Help	3 + 3 tests
15	Groups 12 & 13 — Footer: Social, Copyright	6 + 2 tests
16	Group 14 — Page-wide checks	4 tests
17	Hard vs soft: what green actually proves	The 22 conditional tests, named
18	Artifacts and run variants	Reports, screenshots, grep recipes
19	Failure modes and fixes	Troubleshooting reference

How to read this document

Sections 6 through 16 walk the script in file order, one table per `describe` group. Each row gives the **action**, the exact **locator**, and the **assertion** — plus a strength marker saying whether the test can actually fail:

Marker	Meaning
hard	An `expect()` always runs. If the element is missing or wrong, the test fails.
soft	The assertion sits inside `if (await …isVisible())`. A missing element means the test passes having checked nothing.
log	No `expect()` at all — the test records a value to the console for a human to read.

Section 17 collects every soft and log test in one place, so you can see at a glance what a green run does and does not guarantee.

1 What this script does

It is the shop-window check: everything a visitor sees *before* they have an account, verified element by element, in whichever language the portal happens to be serving.

The public home page at `/public/home` is the only page most first-time visitors ever see. It carries the value proposition, the live platform statistics, the course carousels, the partner credibility block, the explainer videos and the entire footer — and every one of those is a separate conversion risk if it breaks. This spec walks the page top to bottom and checks each element individually.

It needs no login, creates nothing, and changes no server state. That makes it the cheapest suite in the project to run and the safest to run often.

The four things it verifies

Presence

- Logo, headings, subtexts
- The four platform statistics with their exact numbers
- Section headings down the whole page
- Footer columns and copyright

Navigation

- Log in opens a modal, Escape closes it
- Create Account and Start Today navigate
- Both View All links navigate
- Course cards, Privacy policy, Terms of Use

Integrity

- No broken images anywhere on the page
- No JavaScript console errors
- Page loads in under 10 seconds
- Page title names Sphere or Aastrika

Bilingual by design

This is the defining constraint of the suite. `sphere.aastrika.org` serves the public page in **Hindi by default**, but the same build can render English. A suite written against English strings would fail wholesale on the live site; one written against Hindi would fail on a rebuilt English environment.

So every text assertion in the file goes through one helper, `tr()`, which matches **either** language. Section 3 explains how it works. Labels that render identically in both languages — course titles, partner names, numbers, email addresses — are matched directly.

Entry points

Command	What it runs
`npm run run:public`	All 64 tests, headed, against staging. The full walkthrough this document describes.
`npm run report:ci`	This spec plus `public-login.spec.ts`, headless on the Chrome project — the public-surface CI gate.
`npx playwright test tests/PublicPage.spec.ts --grep "Footer"`	Just the four footer groups. Any group name works as a grep target.

> **Run it from the right folder**
> Every Playwright command must be issued from `SphereWebAutomation\`, not the `D:\Automation` root.

2 Configuration and inputs

2.1 How the target URL is built

The spec takes an *origin* and appends the path itself, defending against two ways the environment variable is commonly mis-set:

```
// tests/PublicPage.spec.ts:16
const BASE_URL = (process.env.BASE_URL ?? 'https://sphere.aastrika.org')
  .replace(/\/+$/, '')              // strip trailing slashes
  .replace(/\/public\/home$/, '');   // strip an already-appended path

const PUBLIC_URL = `${BASE_URL}/public/home`;
```

BASE_URL as supplied	PUBLIC_URL that results
(unset)	https://sphere.aastrika.org/public/home
https://sphere.aastrika.org/	https://sphere.aastrika.org/public/home
https://sphere.aastrika.org/public/home	https://sphere.aastrika.org/public/home

Without the second `replace`, the third row would produce `/public/home/public/home` — a 404 that would fail all 64 tests with a misleading "element not found" on every one.

2.2 Runner settings inherited from playwright.config.ts

Setting	Value	Why it matters here
fullyParallel	false	Tests run one at a time.
workers	1	A single browser tab at a time, so a run is a readable linear log. It also means 64 tests each reloading the page is the dominant cost — see the timing note below.
retries	0	A failure is reported as a failure, not retried away. A genuinely flaky element shows up as flaky.
timeout	300 s	Per-test ceiling. No test here comes close; the real limit is the 10 s expect timeout.
expect.timeout	10 s	How long every `toBeVisible()` polls before failing. This is the number that matters on a slow-loading carousel.
screenshot	'on'	A screenshot is captured for **every** test, pass or fail — so the report doubles as a visual record of the whole page.
video	retain-on-failure	Video is kept only for failures.

> **Why the suite takes a while for a read-only check**
> `beforeEach` navigates to the public page afresh for all 64 tests, and eleven of them navigate away and come back. With one worker that is roughly 75 full page loads per run. It is still far cheaper than any signup suite, but it is not instant — `--grep` to a group when iterating.

2.3 The two scroll helpers

Helper	Line	Behaviour
scrollAndClick	:74	`scrollIntoViewIfNeeded()` then `click()`, both unguarded. Used where the element is expected to exist — a missing one should fail loudly.

Helper	Line	Behaviour
`safeScroll`	`:81`	Scrolls with a 3 s timeout, swallows the error, and **returns whether the element is visible**. Used for elements that legitimately vary between builds — partner logos, collaboration marks, social icons. Without it, one absent logo costs a 15 s `scrollIntoViewIfNeeded` timeout and a red test for something that was never guaranteed.

```
// :81 — returns a boolean instead of throwing
async function safeScroll(locator): Promise<boolean> {
  await locator.scrollIntoViewIfNeeded({ timeout: 3000 }).catch(() ⇒ {});
  return locator.isVisible().catch(() ⇒ false);
}
```

`safeScroll` is the single mechanism behind most of the suite's soft tests. It is a deliberate trade — fewer false failures, at the cost of a green result that may have checked nothing. Section 17 lists exactly where that applies.

3 The bilingual engine

One helper, one lookup table, and every text assertion in the file becomes language-agnostic.

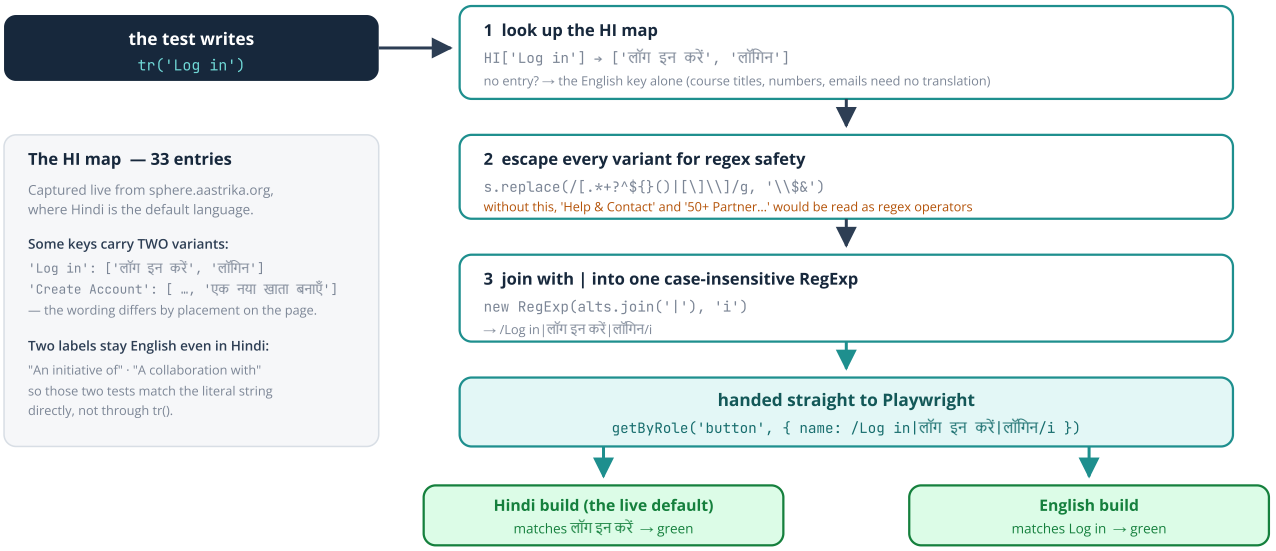


Figure 1 — `tr()` turns one English key into a regex that matches either language, so the same assertion is green on a Hindi build and an English one.

The whole helper, in six lines

```
// :66
function tr(en: string): RegExp {
  const esc  = (s: string) ⇒ s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
  const alts = [en, ...(HI[en] || [])].map(esc);
  return new RegExp(alts.join('|'), 'i');
}
```

Why each of the three steps is load-bearing

1. **The English key always comes first** in the alternation. That is what makes an unmapped label work — `tr('Normal Labour & Birth')` has no HI entry, so it degrades to a plain English match rather than throwing.
2. **Escaping is not optional.** Real labels on this page contain `&`, `+`, `?`, `!` and `.` — "Help & Contact", "50+ Partner Organizations and Growing", "New User?", "Transform your Healthcare career through Sphere!". Unescaped, `50+` would mean "one or more zeroes" and would not match the text on screen.
3. **The `i` flag** lets `tr('TOP COURSES')` match a heading the CSS may render in any case, and covers casing drift between builds.

> **Adding a label**
> When a new string appears on the page, add one entry to the `HI` map at line 27 and use `tr('Your English Label')` in the test. Nothing else changes. If the label happens to render identically in both languages, skip the map entry entirely — `tr()` already handles that case.

4 The execution model

Unlike the signup suite, nothing here is serial. Every test starts from a clean page and is independent of every other.

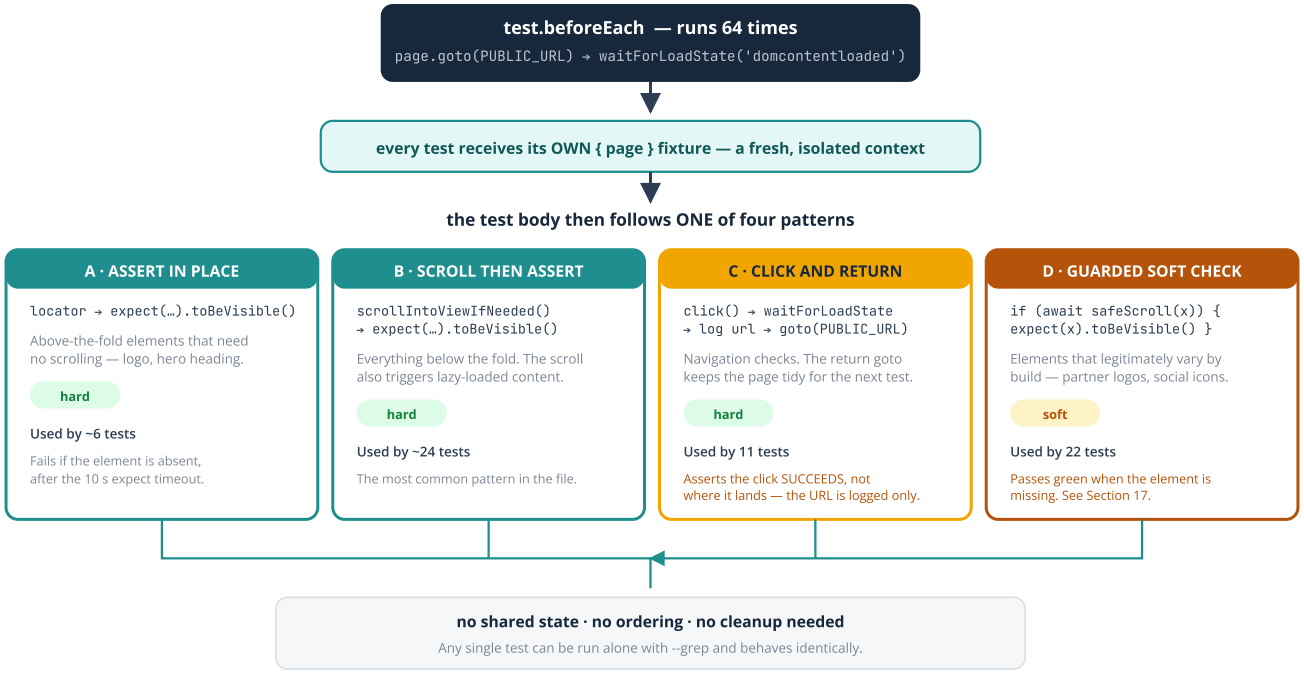


Figure 2 — One `beforeEach` navigation per test, then one of four body shapes. The four patterns account for all 64 tests.

The trade this model buys

Re-navigating for every test is slower than walking the page once, but it makes each test a standalone statement about one element. A failure names exactly one thing, no test can corrupt another's starting state, and `--grep` on any single test name reproduces the failure precisely. On a read-only page with no server cost, that is the right trade.

5 Page anatomy

All 13 element groups mapped onto the page they cover, in the order the script declares them.

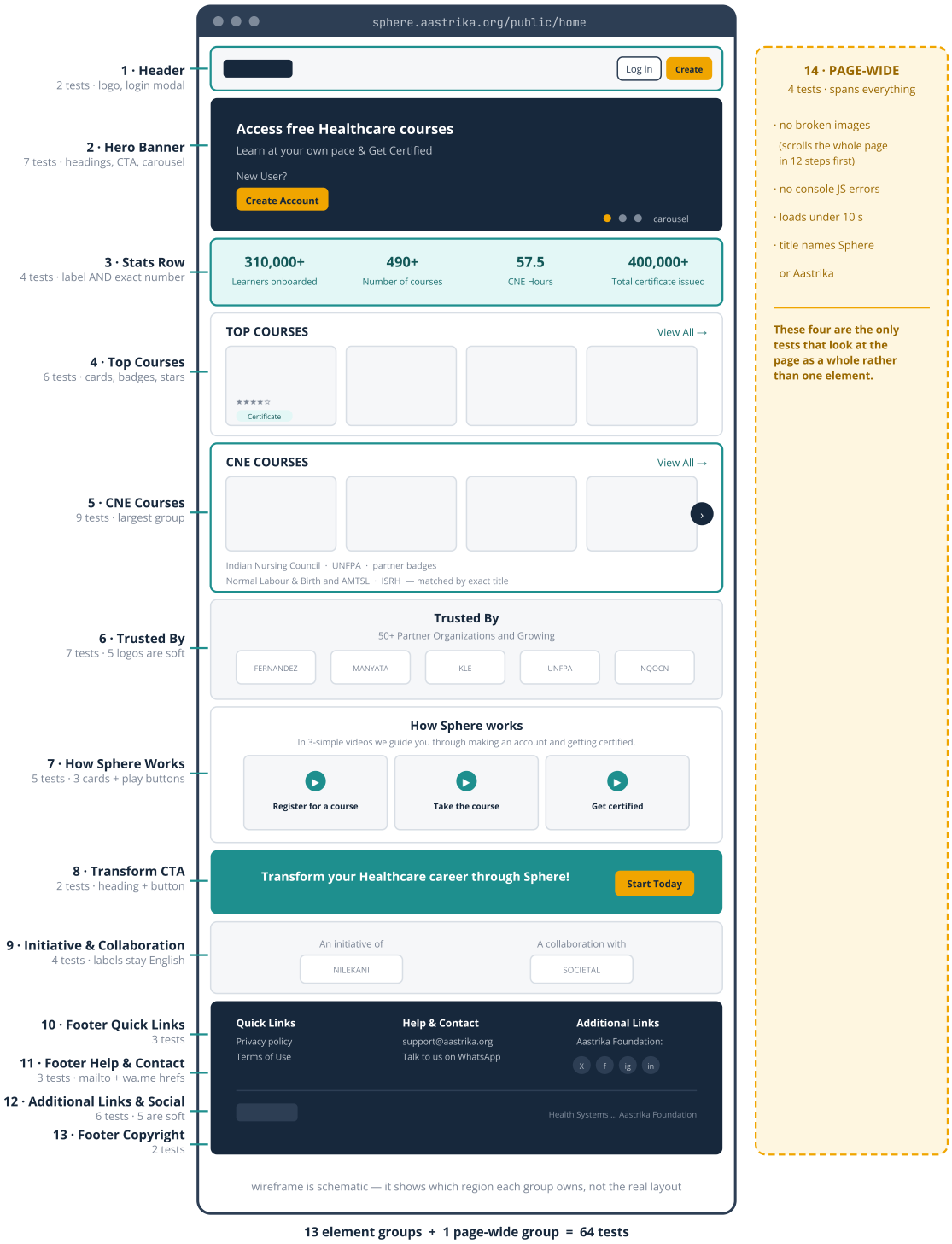


Figure 3 — Every `describe` group mapped to the region of the public home page it covers. Group order in the file follows the page top to bottom.

6 Group 1 — Header

`Header` · 2 tests · line 99

What it proves: the site is branded, and the login entry point opens and closes cleanly without leaving a modal stuck over the page.

#	Action	Locator	Assertion
1	**Logo is visible** Three alternatives in one locator, so a markup change in any one place does not break it.	`img[alt*="logo"], [class*="logo"], ws-app-public-nav-bar img → .first()`	hard `toBeVisible()`. Covers an alt-text match, a class match, and the Angular nav-bar component's own image.
2	**Log in opens a modal, Escape closes it** Four steps in one test: assert, click, assert modal, dismiss.	`getByRole('button', { name: tr('Log in') }) then [class*="modal"], [class*="login"], [role="dialog"] then keyboard.press('Escape')`	hard button visible **and** enabled; modal visible within 5 s. The Escape press and 500 ms wait are not asserted — the test's value is that the modal opened at all.

Why the modal locator is deliberately loose

`[class*="modal"], [class*="login"], [role="dialog"]` matches on any of three signals. The login overlay's markup has changed between builds; an assertion pinned to one class name would have to be rewritten each time, while "something modal-shaped appeared" stays true. The cost is that a wrongly-styled element could satisfy it — acceptable for a smoke check whose real subject is the click.

7 Group 2 — Hero Banner

`Hero Banner` · 7 tests · line 130

What it proves: the value proposition renders, the primary call to action navigates, and the rotating carousel responds — with the rotation itself handled as a soft check, because a slide that is not currently showing is not a defect.

#	Action	Locator	Assertion
1	**Hero heading**	`getByText(tr('Access free Healthcare courses')).first()`	hard visible. Pattern A — above the fold, no scroll.
2	**Hero subtext**	`getByText(tr('Learn at your own pace & Get Certified')).first()`	hard visible. The `&` is why `tr()` escapes before building the regex.
3	**"New User?" prompt**	`getByText(tr('New User?')).first()`	hard visible. The `?` would be a regex quantifier unescaped.
4	**Create Account navigates** Pattern C — clicks, logs where it went, returns.	`getByRole('button', { name: tr('Create Account') }).first()` then `goto(PUBLIC_URL)`	hard visible **and** enabled, and the click must succeed. The destination URL is logged, not asserted.
5	**CNE Hours banner text** Lives on a rotating slide that may not be active.	`getByText(/check out courses with cne hours|सीएनई घंटों वाले पाठ्यक्रम/i)`	log — records `visible: true/false` and always passes. Bilingual regex written inline rather than through `tr()`.
6	**"View courses" button** Same slide, so same conditionality.	`getByRole('button', {name:/view courses|पाठ्यक्रम देखें/i}).or(getByText(…))`	soft — if visible, scrollAndClick then return to the page; if not, logs "not on current banner slide — skipping".
7	**Carousel dots** Clicks dot 2, then dot 1, proving the indicators are interactive.	`[class*="dot"], [class*="carousel-indicator"], [role="tab"]`	soft — only runs `if (count > 1)`. No `expect()`; the count is logged.

The carousel is the reason three of these seven are soft

The hero rotates automatically. Tests 5, 6 and 7 target content that exists only on a particular slide, so a hard assertion would fail roughly whenever the timing was unlucky — the classic flaky test. The script chooses to log instead. That is the right call for a rotating element, but it does mean a broken CNE slide would not turn this suite red. If that slide matters, the fix is to pin the carousel to a known slide first, not to harden the assertion as written.

8 Group 3 — Stats Row

`Stats Row` · 4 tests · line 205

What it proves: the four platform statistics render — both the translated label and the exact number beside it. These are the strongest assertions in the file: each test makes two hard checks and both must hold.

#	Statistic	Locators (two per test)	Assertion
1	**Learners onboarded** 310,000+	`getByText(tr('Learners onboarded')).first()` `getByText('310,000+').first()`	hard ×2 — scroll to the label, assert it, then assert the literal number string.
2	**Number of courses** 490+	`getByText(tr('Number of courses')).first()` `getByText('490+').first()`	hard ×2.
3	**CNE Hours** 57.5	`getByText(tr('CNE Hours')).first()` `getByText('57.5').first()`	hard ×2.
4	**Total certificate issued** 400,000+	`getByText(tr('Total certificate issued')).first()` `getByText('400,000+').first()`	hard ×2.

All four follow pattern B identically: scroll the label into view, assert it, then assert its number. The numbers are matched as plain strings, not through `tr()` — digits and separators render the same in both languages.

These four tests are hard-coded to today's numbers

`310,000+`, `490+`, `57.5` and `400,000+` are literals in the spec. They are marketing figures on a live platform: the moment the site updates a counter, the matching test goes red on a page that is working perfectly. That is a maintenance cost, not a bug — but it is worth knowing where it lives. If these begin to churn, the durable alternative is to assert the label plus a numeric shape (`/[\d,]+\+/`) rather than the exact value.

Where to change them

```
// :211, :220, :229, :238 — one literal per test
const number = page.getByText('310,000+').first();   // Learners onboarded
const number = page.getByText('490+').first();       // Number of courses
const number = page.getByText('57.5').first();       // CNE Hours
const number = page.getByText('400,000+').first();   // Total certificate issued
```

9 Group 4 — Top Courses

`Top Courses Section` · 6 tests · line 248

What it proves: the first course carousel populates from the content API, its cards carry the badges and ratings a learner uses to choose, and both the section link and an individual card navigate.

#	Action	Locator	Assertion
1	**"TOP COURSES" heading**	`getByText(tr('TOP COURSES')).first()`	hard scroll then visible.
2	**"View All" navigates** Takes the *first* View All on the page — this section's.	`getByText(tr('View All')).first()` then `goto(PUBLIC_URL)`	hard visible, and the click must succeed. Destination logged.
3	**Course cards render** The Angular component name first, a class fallback second.	`ws-web-course-card, [class*="course-card"]`	hard `count > 0`. An empty carousel — the content API failing — fails here.
4	**Certificate badges** The one test in the group that waits properly for lazy content.	`getByText(/Certificate|प्रमाणपत्र/i)`	hard `toBeVisible({ timeout: 15000 })` on the first badge, **then** `count > 0`.
5	**Star ratings**	`[class*="rating"], [class*="star"]`	hard `count > 0` — an instantaneous count with no auto-wait.
6	**First card is clickable**	`ws-web-course-card, [class*="course-card"]` → `.first()` then `goto(PUBLIC_URL)`	hard visible, click must succeed. Destination logged.

Test 4 is the template the others should follow

Course cards are lazy-loaded: the carousel populates from an API call that resolves *after* the scroll. An immediate `count()` is a single snapshot with no retry — call it a moment too early and it returns zero on a page that fills in correctly a second later. That is exactly how an intermittent failure is born.

```
// :264 — the comment in the file says it plainly
// Auto-wait for the (lazy-loaded) cards instead of an instantaneous count()
// that can fire before the carousel populates.
await expect(certBadges.first()).toBeVisible({ timeout: 15000 });
const count = await certBadges.count();
expect(count).toBeGreaterThan(0);
```

Tests 3 and 5 still use the bare `count()` form. They pass because test 4's pattern is not the only thing keeping them honest — the preceding `scrollIntoViewIfNeeded` gives the carousel time to fill — but they are the two most likely places in this group for an intermittent failure to appear. If either starts flaking, copy the `toBeVisible`-then-`count` shape from test 4.

> **Why `ws-web-course-card` comes first in the locator**
>
> It is the Angular component's own tag name — the most stable handle available, because it changes only if the component itself is renamed. `[class*="course-card"]` follows as a fallback for builds where the markup differs. Ordering the specific selector first and the loose one second is the pattern used throughout this file.

10 Group 5 — CNE Courses

`CNE Courses Section` · 9 tests · line 311 · 6 of 9 are soft or log

What it proves: the accredited-course carousel renders and is navigable. CNE courses carry continuing-education credit, so the section also shows accrediting-body badges — content that varies, so checked softly.

#	Action	Locator	Assertion
1	**"CNE COURSES" heading**	`getByText(tr('CNE COURSES')).first()`	hard scroll then visible.
2	**Second "View All" navigates** Counts them and takes index 1 — this section's link.	`getByText(tr('View All')) → count > 1 ? .nth(1) : .first()`	hard visible, click must succeed. The `count > 1` ternary degrades to the first link if only one exists.
3	**CNE cards render**	`ws-web-course-card, [class*="course-card"]`	hard `count > 0`.
4	**CNE hours badges**	`[class*="cne"].or(getByText(/CNE:/))`	log — counts and prints. **No `expect()` at all**; zero badges passes.
5	**Carousel right arrow**	`[class*="next"], [aria-label*="next"], button[class*="arrow"] → .last()`	soft — clicks only `if visible`. No assertion either way.
6	**"Normal Labour & Birth and AMTSL" card**	`getByText('Normal Labour & Birth and AMTSL').first()`	soft — clicks and returns only if visible. Course titles are not translated, so no `tr()`.
7	**"Integrated Sexual and Reproductive Health (ISRH)" card**	`getByText('Integrated Sexual and Reproductive Health (ISRH)').first()`	soft — same shape as test 6.
8	**Indian Nursing Council badge**	`getByText('Indian Nursing Council').first()`	soft — `if visible` then assert visible.
9	**UNFPA badge**	`getByText('UNFPA').first()`	soft — same shape as test 8.

The guarded shape, and what it costs

Tests 6 and 7 name real CNE course titles. Matching by exact string is stronger than "some card exists" — it proves the carousel serves *accredited* content rather than the general catalogue. Tests 8 and 9 check the accreditor badges, and all four share one shape:

```
const badge = page.getByText('Indian Nursing Council').first();
if (await badge.isVisible()) {          // ← absent? the body never runs
  await expect(badge).toBeVisible();      // ← can only ever pass
  console.log('✅ Indian Nursing Council badge visible');
}
```

Present, it asserts something true; absent, the test passes silently. The console line prints only on the success branch — so a run log *without* that checkmark is the real signal the element was missing. The same guard means a retired or renamed course stops being checked rather than failing.

> **Reading this group's result honestly**
> A green CNE group guarantees three things: the heading renders, the carousel has at least one card, and View All works. Everything else — hours badges, arrow, the two named courses, both accreditor badges — is advisory. Read the console output, not just the pass count.

11 Group 6 — Trusted By

`Trusted By Section` · 7 tests · line 406 · 5 of 7 are soft

What it proves: the social-proof block renders its heading and partner count. The five individual partner logos are checked softly, because which logos appear is content that legitimately varies between builds and environments.

#	Element	Locator	Assertion
1	**"Trusted By" heading**	`getByText(tr('Trusted By')).first()`	hard scroll then visible.
2	**"50+ Partner Organizations and Growing"**	`getByText(tr('50+ Partner Organizations and Growing')).first()`	hard scroll then visible. The `+` here is the clearest case for `tr()`'s escaping.
3	**Fernandez Foundation**	`getByText('FERNANDEZ').or(img[alt*="Fernandez"]) → .first()`	soft via `safeScroll`.
4	**Manyata**	`getByText('MANYATA').or(img[alt*="Manyata"])`	soft via `safeScroll`.
5	**KLE**	`getByText('KLE').or(img[alt*="KLE"])`	soft via `safeScroll`.
6	**UNFPA**	`img[alt*="UNFPA"], [class*="unfpa"] → .first()`	soft via `safeScroll`. Image-only — no text variant, unlike the others.
7	**NQOCN**	`getByText('NQOCN').or(img[alt*="NQOCN"])`	soft via `safeScroll`.

The text-or-image locator

Four of the five logo tests use the same two-way locator:

```
page.getByText('FERNANDEZ').or(page.locator('img[alt*="Fernandez"]')).first()
```

A partner mark may be rendered as an SVG with a text node, or as a raster image with alt text — and which one it is has changed between builds. Matching either means the test survives that change. The UNFPA test (6) matches only on image selectors, because that mark has no text form.

> **Why softness is defensible here and less so elsewhere**
>
> Partner logos are a content list the business edits. A logo disappearing because a partnership ended is a content change, not a regression, and failing a build over it would train people to ignore the suite. Contrast this with the hero carousel in Group 2, where the content is fixed and only the *timing* is variable — there, softness papers over a real flake. Same mechanism, different justification.

12 Group 7 — How Sphere Works

`How Sphere Works Section` · 5 tests · line 467

What it proves: the three-step explainer renders — heading, subtitle, and all three cards with both their title and their description. Each card test also attempts its video play button, which is soft.

#	Element	Locator	Assertion
1	**"How Sphere works" heading**	`getByText(tr('How Sphere works')).first()`	hard scroll then visible. The Hindi keeps the English brand name: "Sphere कैसे काम करता है".
2	**"In 3-simple videos…" subtitle**	`getByText(tr('In 3-simple videos we guide you through making an account and getting certified.')).first()`	hard scroll then visible. The longest key in the HI map.

3	**Step 1 — Register for a course**
	Title, description, then play button.
		`tr('Register for a course')`
		`tr('Explore various courses and pick the ones you like')`
		`[class*="play"], [aria-label*="play"], button[class*="video"]` → `.first()`
			hard ×2 on the text; soft on the play button — clicked only if visible, then Escape.

4	**Step 2 — Take the course**	`tr('Take the course')`
		`tr('Access the course anytime, at your convenience')`
		play buttons → `.nth(1)`, guarded by `count() ⩾ 2`
			hard ×2 on the text; soft on the play button.

5	**Step 3 — Get certified**	`tr('Get certified')`
		`tr('Receive downloadable and shareable certificates')`
		play buttons → `.nth(2)`, guarded by `count() ⩾ 3`
			hard ×2 on the text; soft on the play button.

Tests 3–5 are hybrids

Each makes two hard text assertions and then attempts one soft interaction. That is a useful shape: the part that is guaranteed — the card's copy — fails loudly, while the part that is fragile — a video player overlay — only logs. The play-button locator is shared across all three tests and indexed by position:

```
const playBtns = page.locator('[class*="play"], [aria-label*="play"], button[class*="video"]');
if (await playBtns.count() ⩾ 2) {        // step 2 needs at least two to exist
  await playBtns.nth(1).click();         // index 1 = the second card's button
  await page.waitForTimeout(1000);
  await page.keyboard.press('Escape');      // dismiss whatever opened
}
```

The count guard is what makes positional indexing safe — `nth(2)` on a page with two buttons would otherwise throw rather than skip.

> **What the Escape press is really for**
> Clicking play may open a modal video player. Because every test gets a fresh page, a stuck overlay could not leak into the next test anyway — but the Escape keeps the end-of-test screenshot (captured for every test, pass or fail) showing the page rather than a video frame. Small thing; it makes the report readable.

13 Groups 8 & 9 — Transform CTA and Initiative

Group 8 — Transform CTA 2 tests · line 547

The closing call to action, and the page's second conversion entry point after the hero.

#	Element	Locator	Assertion
1	**"Transform your Healthcare career through Sphere!"**	`getByText(tr('Transform your Healthcare career through Sphere!')).first()`	hard scroll then visible. The trailing `!` is escaped by `tr()`.
2	**"Start Today" navigates**	`getByRole('button', { name: tr('Start Today') }).first()` then `goto(PUBLIC_URL)`	hard visible **and** enabled, and the click must succeed. Destination logged.

Group 9 — Initiative and Collaboration 4 tests · line 573

The institutional-backing block below the CTA. This group is where the bilingual rule has its one documented exception.

#	Element	Locator	Assertion
1	**"An initiative of" label**	`getByText('An initiative of').first()` — **no `tr()`**	hard scroll then visible.
2	**Nilekani Philanthropies logo**	`getByText('NILEKANI').or(img[alt*="Nilekani"]) → .first()`	soft via `safeScroll`.
3	**"A collaboration with" label**	`getByText('A collaboration with').first()` — **no `tr()`**	hard scroll then visible.
4	**Societal Platform logo**	`getByText('SOCIETAL').or(img[alt*="Societal"]) → .first()`	soft via `safeScroll`.

> **The two labels that stay English**
> "An initiative of" and "A collaboration with" are **not** translated by the portal — they render in English even when the rest of the page is Hindi. The spec says so in a comment at both sites, and matches the literal strings rather than routing them through `tr()`. Adding them to the `HI` map would not break anything (the English alternative comes first in the alternation either way), but it would misrepresent how the page behaves. The comment is there so the next person does not "fix" the apparent inconsistency.

The two logo tests use the same text-or-image locator as the Trusted By partner marks, and are soft for the same reason: which institutional logos appear is editable content.

14 Groups 10 & 11 — Footer: Quick Links and Help

Group 10 — Footer Quick Links 3 tests · line 612

The legal links. Both are genuine navigations, so both use pattern C — click, log the destination, return.

#	Element	Locator	Assertion
1	**"Quick Links" heading**	`getByText(tr('Quick Links')).first()`	hard scroll then visible.
2	**Privacy policy navigates**	`getByText(tr('Privacy policy')).first()` then `goto(PUBLIC_URL)`	hard visible, click must succeed. Destination logged.
3	**Terms of Use navigates**	`getByText(tr('Terms of Use')).first()` then `goto(PUBLIC_URL)`	hard visible, click must succeed. Destination logged.

Group 11 — Footer Help & Contact 3 tests · line 648

The only group that asserts on `href` attributes rather than visible text — because for a contact link, the destination *is* the thing that matters.

#	Element	Locator	Assertion
1	**"Help & Contact" heading**	`getByText(tr('Help & Contact')).first()`	hard scroll then visible.
2	**support@aastrika.org email link** Checks the address *and* that it is actually a mail link.	`getByText('support@aastrika.org').first()` then `getAttribute('href')`	hard ×2 — visible, and `href` matches `/mailto:|support@aastrika/i`. A visually-correct address with a dead link fails here.
3	**"Talk to us on WhatsApp"**	`getByText(tr('Talk to us on WhatsApp')).first()` then `a[href*="wa.me"], a[href*="whatsapp"]`	hard on the text being visible; log on the href — it is fetched with `.catch(() ⇒ null)` and printed, never asserted.

A notable asymmetry between tests 2 and 3

The email test asserts its `href`; the WhatsApp test only logs one. So a broken `mailto:` fails the suite, while a broken or missing `wa.me` link passes it with a `WhatsApp href: null` line in the console. If WhatsApp is a supported contact channel, test 3 is the single cheapest place in this file to add real coverage — the assertion would be one line, mirroring test 2.

15 Groups 12 & 13 — Footer: Social and Copyright

Group 12 — Additional Links & Social Icons

6 tests · line 682 | 5 of 6 soft

#	Element	Locator	Assertion
1	**"Additional Links" heading**	`getByText(tr('Additional Links')).first()`	hard scroll then visible. The only hard test in the group.
2	**Aastrika Foundation link**	`a[href*="aastrika.org"] → .last()`	soft — if visible, asserts `href` contains "aastrika". `.last()` picks the footer occurrence rather than an earlier one on the page.
3	**Twitter / X icon**	`a[href*="twitter"], a[href*="x.com"], img[alt*="Twitter"], img[alt*="twitter"]`	soft — href logged only, never asserted. Covers both the old and new brand domains.
4	**Facebook icon**	`a[href*="facebook"], img[alt*="Facebook"]`	soft — href logged only.
5	**Instagram icon**	`a[href*="instagram"], img[alt*="Instagram"]`	soft — href logged only.
6	**LinkedIn icon**	`a[href*="linkedin"], img[alt*="LinkedIn"]`	soft — href logged only.

Tests 3–6 are identical in shape: `safeScroll`, then read and print the `href`. None asserts. The four exist to surface the social links in the run log, not to gate a build on them.

Group 13 — Footer Copyright

2 tests · line 737

#	Element	Locator	Assertion
1	**Copyright text**	`getByText(/Health Systems.*Aastrika Foundation/i).first()`	hard scroll then visible. Matched by a two-anchor regex rather than the full sentence.
2	**Footer logo**	`footer img, [class*="footer"] img, [class*="footer"] [class*="logo"] → .first()`	soft via `safeScroll`. Three fallbacks: the semantic `<footer>` tag, a footer class, and a logo class inside one.

Why the copyright regex has a gap in the middle

`/Health Systems.*Aastrika Foundation/i` anchors on the first and last few words and lets `.*` absorb everything between. The site's own copyright line contains a typo — the spec's comment notes this without editing around it — and a year that changes annually. Anchoring both ends and ignoring the middle means the test survives the typo being fixed and the year rolling over, while still failing if the line disappears. It is the same principle as matching the T&C page by route rather than by wording: match the structure, not the prose.

16 Group 14 — Page-wide checks

`Page-Wide Checks` · 4 tests · line 760 · all four hard

What it proves: the page as a whole is healthy — no dead images, no JavaScript errors, acceptable load time, correct title. These are the four tests that look at the document rather than at one element, and every one of them can fail.

16.1 No broken images

The most carefully engineered test in the file, and the one worth understanding in detail. Two problems had to be solved: lazy-loaded images never start loading unless scrolled to, and an image mid-flight looks identical to a broken one.

1. **Scroll the entire page in 12 steps.** `window.scrollTo(0, i * innerHeight * 0.9)` with a 400 ms pause each, so every `loading="lazy"` thumbnail enters the viewport and begins fetching. The 0.9 factor overlaps the steps slightly — nothing is skipped between them.
2. **Jump to the bottom** with `scrollTo(0, document.body.scrollHeight)`, catching any content the fixed steps missed on a longer-than-expected page.
3. **Wait for quiet** — `waitForLoadState('networkidle')` (errors swallowed) plus a further 3000 ms, so in-flight image requests finish.
4. **Collect only genuine failures** in the browser, using a three-part filter.
5. **Warn, then assert.** Any broken URLs are logged with `console.warn` before `expect(brokenImgs).toEqual([])` fails — so the report names the exact files.

```
Array.from(document.querySelectorAll('img'))
  .filter((img) => {
    const src = img.currentSrc || img.src;
    if (!src || src.startsWith('data:')) return false;  // skip inline data URIs
    return img.complete && img.naturalWidth === 0;        // FINISHED, but has no pixels
  })
```

The `complete && naturalWidth === 0` pair is the whole trick. `complete` means the browser has finished trying; `naturalWidth === 0` means it got nothing. Together they mean a real 404 or a decoding failure. An image still downloading reports `complete: false` and is ignored, as is a lazy image that never scrolled into view — so the test cannot fail on timing alone.

16.2 No console errors

Attaches a `console` listener, reloads the page, and collects everything of type `error`. It then filters the list before asserting:

```
const jsErrors = errors.filter((e) => !/Failed to load resource/i.test(e));
expect(jsErrors).toEqual([]);
```

Resource 404s are excluded deliberately — a missing analytics beacon or a third-party font would otherwise fail a page whose own code is fine, and broken *images* already have their own dedicated test above. What survives the filter is real JavaScript: uncaught exceptions, failed promise rejections, framework errors. Those fail the test, and are printed first with `console.warn`.

16.3 Page loads within 10 seconds

```
const start = Date.now();
await page.reload();
```

```
await page.waitForLoadState('domcontentloaded');
expect(Date.now() - start).toBeLessThan(10000);
```

The elapsed time is printed on every run, pass or fail, which makes this the file's de facto performance trend line — the number in the log is useful even when the test is green. Note the measurement stops at `domcontentloaded`, not at full load: it covers HTML and blocking scripts, not lazy images or carousel API calls. It is a "did the page get going" check, not a complete page-weight budget.

16.4 Page title

```
expect(await page.title()).toMatch(/sphere|aastrika/i);
```

Cheap, and a genuinely useful canary: a wrong or empty title usually means the router served the wrong shell, a deploy landed on the wrong environment, or an error page is being returned with a 200. The title is logged either way.

> **Run this group first when triaging**
> If the whole suite is red, `--grep "Page-Wide"` answers "is the page fundamentally broken, or did one element move?" in four tests and about twenty seconds. A failing title or a flood of console errors explains the other sixty failures; all four green means the problem is a selector, not the page.

17 Hard vs soft: what green actually proves

22 of the 64 tests can pass without checking anything. Here is exactly which, so a green report can be read accurately.

A soft test is one whose assertion sits behind an `if (await …isVisible())` or `if (count ⩾ n)` guard. When the element is present it asserts something real; when it is absent the body never runs and the test reports green. A log test has no `expect()` at all. Neither is a defect — both are deliberate choices against flakiness — but both change what a pass means.

Group	Tests	Hard	Soft/log	What is actually guaranteed
1 · Header	2	**2**	0	Fully guaranteed.
2 · Hero Banner	7	**4**	3	Headings and Create Account. The CNE slide and carousel dots are advisory.
3 · Stats Row	4	**4**	0	Fully guaranteed — all four labels and all four exact numbers.
4 · Top Courses	6	**6**	0	Fully guaranteed.
5 · CNE Courses	9	**3**	6	Heading, cards exist, View All works. Badges, arrow and named courses are advisory.
6 · Trusted By	7	**2**	5	Heading and partner count. All five logos are advisory.
7 · How Sphere Works	5	**5**	0	All eight text elements are guaranteed; only the play buttons are advisory *within* otherwise-hard tests.
8 · Transform CTA	2	**2**	0	Fully guaranteed.
9 · Initiative	4	**2**	2	Both labels. Both logos are advisory.
10 · Footer Quick Links	3	**3**	0	Fully guaranteed.
11 · Footer Help & Contact	3	**3**	0	Heading, email text + mailto href, WhatsApp text. The wa.me href itself is logged only.
12 · Additional & Social	6	**1**	5	The heading. All four social icons and the foundation link are advisory.
13 · Footer Copyright	2	**1**	1	The copyright line. The footer logo is advisory.
14 · Page-Wide	4	**4**	0	Fully guaranteed — images, console, load time, title.
Total	**64**	**42**	**22**	**Roughly two thirds of the suite gates a build; one third reports.**

How to read a run

The soft tests still produce evidence — they just put it in the console instead of the exit code. Their `console.log` lines print **only on the success branch**, so absence of a line is the signal:

```
✅ Fernandez Foundation logo visible
✅ Manyata logo visible
                                ← no KLE line: the KLE logo was not found
✅ UNFPA logo visible
✅ NQOCN logo visible
```

All five tests report green. Only the log says one logo is missing. When this suite matters, read the output, not just the count.

> **Where hardening would pay, if you want it**

Three places stand out, in order of value. **The WhatsApp href** (Group 11, test 3) — a one-line assertion mirroring the email test, covering a live contact channel. **The CNE hours badge** (Group 5, test 4) — it has no `expect()` at all today, and CNE credit is the section's entire purpose. **The social icons** (Group 12) — at minimum assert that *some* href was found, rather than printing `null`. The partner and institutional logos are best left soft: that list is editable content, and failing a build over a business decision is how a suite gets ignored.

18 Artifacts and run variants

18.1 What a run leaves behind

Artifact	Contents
`test-results/`	**A screenshot for every one of the 64 tests** — the config sets `screenshot: 'on'`, not `only-on-failure`. A passing run therefore leaves a complete visual record of the page, section by section — useful for eyeballing a deploy even when everything is green.
`test-results/…/video.webm`	Video, kept only for failures (`retain-on-failure`).
`playwright-report/`	The HTML report — the normal way to read a run.
`console output`	Every navigation destination, every found `href`, every soft-check result, and the load time in ms. For this suite the log carries information the pass/fail count does not.

18.2 A healthy run, abridged

```
✅ Page loaded: https://sphere.aastrika.org/public/home
✅ Login button clicked → ✅ Login modal opened → ✅ Login modal closed
✅ Hero heading visible
✅ Create Account button clicked → URL: …/app/create-account
ℹ️  "View courses" not on current banner slide — skipping
✅ Learners onboarded stat visible
✅ Found 12 course cards
✅ Found 8 Certificate badges
✅ View All clicked → URL: …/app/search
✅ Email link visible with href: mailto:support@aastrika.org
   WhatsApp href: https://wa.me/91…
✅ No broken images
✅ No console JS errors (resource 404s ignored)
✅ Page loaded in 2184ms
✅ Page title: Aastrika Sphere
```

18.3 Run variants

```bash
# the whole suite, headed, against staging
npm run run:public

# one group — any describe name works as a grep target
npx playwright test tests/PublicPage.spec.ts --grep "Footer"
npx playwright test tests/PublicPage.spec.ts --grep "Page-Wide"      # triage first
npx playwright test tests/PublicPage.spec.ts --grep "CNE Courses"

# one test, by its exact name
npx playwright test tests/PublicPage.spec.ts --grep "No broken images"

# point at another environment (the origin is normalised for you)
cross-env BASE_URL=https://portal-staging.example.org \
  npx playwright test tests/PublicPage.spec.ts --headed

# headless, with the public-login suite — the CI gate
npm run report:ci

# open the report / a trace
npx playwright show-report
npx playwright test tests/PublicPage.spec.ts --trace on
```

19 Failure modes and fixes

Symptom	Most likely cause	What to do
All 64 tests fail on "element not found"	The page never loaded — usually a bad `BASE_URL`, or Playwright run from the repo root instead of `SphereWebAutomation\`.	Run `--grep "Page-Wide"` first. A failing title test confirms the page itself is wrong; check the URL printed by `beforeEach`.
Every text assertion fails, but the page looks fine	The portal is serving a language variant the `HI` map does not cover.	Open the page, copy the actual rendered strings, and add them to the map at line 27. `tr()` accepts an array, so add a variant rather than replacing one.
One of the four stat tests fails	Almost certainly not a bug — the platform's live counter moved past the hard-coded literal.	Update the literal at line 211/220/229/238. If they churn often, switch the number assertion to a shape match such as `/[\d,]+\+/`.
"Found 0 course cards" / card count fails	The content API did not return in time, or genuinely returned nothing.	Check whether the Certificate-badge test (which waits up to 15 s) also failed. If it passed and the count test did not, it is the instantaneous `count()` — give tests 3 and 5 the same `toBeVisible`-then-`count` shape.
Broken-images test lists thumbnails	Real 404s in the course-thumbnail CDN, or images that failed to decode.	The failure message names every URL. Open one directly — the lazy-loading and in-flight cases are already filtered out, so anything listed genuinely returned nothing.
Console-errors test fails	A genuine uncaught JavaScript error — resource 404s are already filtered.	Read the `console.warn` line printed before the failure; it carries the exact messages. This one is rarely a test problem.
Load-time test fails intermittently	The 10 s budget is measured to `domcontentloaded`; a cold CDN or a slow network can exceed it.	Check the logged milliseconds across recent runs before changing anything. A steady creep is a real regression; one outlier is not.
A logo or social test passes but you know the element is gone	Working as designed — it is one of the 22 soft tests.	Check the console for the missing ✅ line. If that element must gate the build, remove its `if (await safeScroll(...))` guard and let the assertion run unconditionally.
"View All" clicks the wrong section	The two View All links are addressed by position — `.first()` for Top Courses, `.nth(1)` for CNE.	If a third carousel is added, or the sections are reordered, these indices need revisiting. Scope the locator to the section container instead of indexing the whole page.
Hero carousel tests log "skipping" every run	The rotation timing never lands on that slide while the test looks.	Expected behaviour today. To make it deterministic, click the relevant carousel dot to pin the slide before asserting, rather than hardening the assertion as written.

The one rule to keep

Every text assertion in this file goes through `tr()`, and every element locator names two or three alternatives. Both habits exist for the same reason: this page's **language, content and markup all change independently of its behaviour**. When you add a check, match the structure and accept either language — and when you make something hard rather than soft, do it because the element is guaranteed, not because green feels better.